Version Control Tools: Git and GitHub

Concepts, Commands, and Collaborative Development

Prof. Neeta Nain

MNIT Jaipur
Department of Computer Science and Engineering

October 12, 2026

Learning Objectives

By the end of this lecture, students will be able to:

- Explain the need for version control.
- Understand Git and GitHub.
- Create and manage Git repositories.
- Use essential Git commands.
- Upload and maintain projects on GitHub.
- Collaborate through branches and pull requests.

What Is Version Control?

Definition

Version control is a system that records changes to files over time so that previous versions can be inspected, compared, and recovered.

Key capabilities:

- Track modifications to files.
- Identify who made a change and when.
- Restore earlier versions when necessary.
- Support multiple contributors.

Why Do We Need Version Control?

Consider a project shared by several developers.

- Files may be overwritten accidentally.
- Previous working versions may be lost.
- Identifying the source of an error can be difficult.
- Combining changes from different contributors can become complicated.

Key Idea

Version control makes changes traceable and project history manageable.

Life Without Version Control

A Common Problem

- project.py
- project_final.py
- project_final_new.py
- project_final_latest.py
- project_final_latest_fixed.py

Problems:

- Which file contains the latest correct version?
- What changed between two files?
- How can an earlier version be recovered?

Benefits of Version Control

For Developers

- Change tracking
- Error recovery
- Branch-based development
- Better collaboration

For Projects

- Organized history
- Accountability
- Reproducibility
- Easier maintenance

Types of Version Control Systems

1. **Local VCS**
   - Versions are managed on one computer.
2. **Centralized VCS**
   - A central server stores the project history.
   - Example: Subversion (SVN).
3. **Distributed VCS**
   - Each clone contains repository history.
   - Example: Git.

Git: An Introduction

Definition

Git is a distributed version control system used to track changes in files and coordinate development.

Important features:

- Maintains project history through commits.
- Supports parallel development using branches.
- Enables local version control without a network.
- Supports merging contributions from different branches.

Git vs GitHub

Feature	Git	GitHub
Type	Version control tool	Hosting and collaboration platform
Location	Works locally	Provides remote repositories
Purpose	Tracks project changes	Shares and reviews projects
Network	Local work is possible offline	Online features require connectivity

Remember

Git is the version control system; GitHub hosts Git repositories and provides collaboration features.

Real-World Applications

- **Software Engineering:** Managing source code and releases.
- **Team Projects:** Combining work from multiple developers.
- **Research:** Tracking experimental code and configuration changes.
- **Machine Learning:** Versioning training scripts and documentation.
- **Documentation:** Maintaining LaTeX reports and technical manuals.

Transition

Next: Git installation, configuration, repositories, and the basic Git workflow.

How Git Works

Git records project changes as a sequence of commits.

Version 1 (Initial Code) $\longrightarrow$ **Version 2** (Bug Fix) $\longrightarrow$ **Version 3** (New Feature)

How Git Manages Changes

- Files are modified in the working directory.
- Selected changes are added to the staging area.
- A commit records the staged changes.
- Git maintains a history that can be inspected later.

Key Concept

A commit represents a recorded project snapshot, not simply a backup of an individual file.

Git Architecture

Git supports local development and collaboration through remote repositories.



- **Working directory:** Files being edited.
- **Local repository:** Local commits and history.
- **Remote repository:** Shared repository hosted on a server.

Installing Git

Git can be installed on Windows, Linux, and macOS.

Windows

Download Git from the official website: `https://git-scm.com/downloads`

Ubuntu / Debian Linux

```
sudo apt update
sudo apt install git
```

macOS

Install using Homebrew:

```
brew install git
```

After installation: Open a terminal or Git Bash to execute Git commands.

Verifying Git Installation

After installation, check whether Git is available.

Command

```
git --version
```

Example output:

```
git version 2.x.x
```

- The version number depends on the installed release.
- If the command is not recognized, check the installation and system PATH.
- Run the command in a terminal, Git Bash, or an integrated development environment terminal.

Configuring Git

Git uses configuration values to identify the author of commits.

Set Username

```
git config --global user.name "Student Name"
```

Set Email

```
git config --global user.email "student@example.com"
```

Verify Configuration

```
git config --global --list
```

These values identify the commit author. They do not authenticate the user to GitHub.

What Is a Git Repository?

A Git repository stores project history and the information needed to track changes.

Project Files

- main.py
- README.md
- data.csv
- requirements.txt

Git Metadata

- Commit history
- Branch information
- Object database
- Repository configuration

Observation

Git stores its repository metadata in the hidden `.git` directory.

Creating a Git Repository

Follow these steps to initialize a project.

1. Create a project directory.
2. Open a terminal inside that directory.
3. Initialize Git.

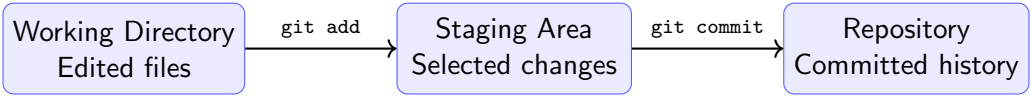
Commands

```
mkdir my-project
cd my-project
git init
```

- `mkdir` creates a directory.
- `cd` changes the current directory.
- `git init` initializes a Git repository.

Result: Git creates a repository in the current directory.

The Three Main Areas of Git



- **Working directory:** Where files are edited.
- **Staging area:** Where changes are selected for the next commit.
- **Repository:** Where committed project history is stored.

Understanding a Git Commit

A commit records a set of staged changes in the repository.

Example

```
git add main.py
git commit -m "Add main program"
```

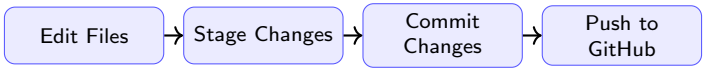
Each commit includes information such as:

- A unique identifier (hash).
- Author and timestamp.
- A commit message.
- References to the recorded project snapshot and its parent commit or commits.

Best Practice

Write meaningful commit messages that explain the purpose of the change.

The Basic Git Workflow



Commands to Remember

1. `git status` – Inspect repository status.
2. `git add .` – Stage changes.
3. `git commit -m "message"` – Record changes.
4. `git push` – Upload commits to a remote.

Note: Configure a remote before pushing to GitHub.

Checking Repository Status

The `git status` command displays the current state of the working directory and staging area.

Command

```
git status
```

It helps identify:

- Untracked files.
- Modified files.
- Staged changes ready to commit.
- Whether the working tree is clean.

Example

After creating a new file, Git reports it as untracked until it is added to the staging area.

Tracking Files with Git

Use `git add` to stage a new file or its changes.

Command

```
git add main.py
```

Workflow:

1. Create or modify `main.py`.
2. Run `git status`.
3. Stage the file using `git add main.py`.
4. Run `git status` again.

Important

`git add` stages the file's current contents. Changes made after staging must be staged again.

Staging Multiple Changes

Git can stage several files with a single command.

Stage All Changes in the Current Directory

```
git add .
```

Other useful commands:

```
git add file1.py file2.py
git add *.py
```

- Stage selected files when changes need to be separated.
- Review staged changes before committing.
- Use `git diff --staged` to inspect staged changes.

Best Practice

Avoid staging sensitive files or generated files accidentally. Check `git status` before committing.

Creating a Git Commit

A commit records staged changes in the local repository.

Command

```
git add main.py
git commit -m "Add main program"
```

A good commit message should:

- Describe the purpose of the change.
- Be concise and specific.
- Avoid vague messages such as “changes” or “update”.

Examples

- `Add input validation`
- `Fix division by zero error`
- `Update project documentation`

Viewing Commit History

Use `git log` to inspect previous commits.

Command

```
git log
```

Git displays information such as:

- Commit hash.
- Author.
- Date.
- Commit message.

Why Is It Useful?

Commit history helps developers understand how a project evolved and identify when a change was introduced.

Viewing a Compact Commit History

For a shorter history, use `git log --oneline`.

Command

```
git log --oneline
```

Example output:

```
a1b2c3d Add input validation
d4e5f6a Fix calculation bug
b7c8d9e Initial project commit
```

- Each line represents one commit.
- The abbreviated hash identifies the commit.
- The message summarizes the change.

Comparing Changes with Git

The `git diff` command helps identify differences between versions of files.

Useful Commands

```
git diff
git diff --staged
git diff HEAD
```

- `git diff`: Changes not yet staged.
- `git diff --staged`: Changes staged for commit.
- `git diff HEAD`: Changes relative to the current commit, including staged and unstaged tracked changes.

Use Case

Inspect changes before committing to catch unintended edits.

Inspecting Tracked Files

The `git ls-files` command lists files known to Git in the index.

Command

```
git ls-files
```

Example output:

```
README.md
main.py
requirements.txt
```

- Useful for inspecting files tracked by Git.
- Untracked files are not normally shown.
- A file listed here may have staged changes that are not yet committed.

Ignoring Files with .gitignore

The `.gitignore` file specifies intentionally untracked files and directories that Git should ignore.

Example .gitignore

```
__pycache__/
*.pyc
.env
*.log
data/
```

Typical uses:

- Temporary files and caches.
- Local environment configuration.
- Logs and generated outputs.
- Large datasets that should not be committed.

Renaming and Deleting Files

Git provides commands for recording file moves and deletions.

Rename or Move a File

```
git mv old.py new.py
```

Remove a Tracked File

```
git rm old.py
```

- `git mv` stages the rename operation.
- `git rm` stages the removal and removes the file from the working directory.
- Git often detects renames by comparing file contents; it does not store renames as a separate fundamental object type.

Unstaging Changes

If a file was staged accidentally, remove it from the staging area without discarding its working-directory changes.

Command

```
git restore --staged main.py
```

What happens?

- The staged version is removed from the index.
- The file's working-directory edits are preserved.
- The file may appear as modified or untracked.

Remember

Unstaging a file is different from deleting its changes.

Discarding Uncommitted Changes

The command below restores a tracked file from the index and discards its unstaged modifications.

Command

```
git restore main.py
```

Warning

Unstaged changes discarded by this command may be difficult or impossible to recover.

Before running it:

- Inspect changes using `git diff`.
- Save important work elsewhere if needed.
- Understand whether the changes are staged or unstaged.

Reverting a Commit Safely

The `git revert` command creates a new commit that reverses the changes introduced by an earlier commit.

Command

```
git log --oneline
git revert COMMIT_ID
```

- Existing commit history is preserved.
- A new commit records the reversal.
- It is often suitable for undoing published changes.

Example

If a commit introduced a bug, reverting it can undo its changes without deleting the original commit from history.

Complete Local Git Example

Create a small project and record its first version.

Commands

```
mkdir calculator
cd calculator
git init

echo "print(2 + 3)" > main.py

git status
git add main.py
git commit -m "Add calculator program"

git log --oneline
```

- Initialize the repository.
- Create a Python file.
- Stage and commit the file.
- Inspect the recorded history.

Next

GitHub and remote repositories: publishing and sharing projects.

What Is GitHub?

Definition

GitHub is an online platform for hosting Git repositories and collaborating on software projects.

Major features:

- Host and share source code.
- Maintain project history through Git.
- Collaborate with other developers.
- Review code using pull requests.
- Track bugs and tasks using Issues.

Remember

Git manages version history; GitHub provides hosting and collaboration features around Git repositories.

Git vs GitHub

Feature	Git	GitHub
Type	Version control system	Hosting and collaboration platform
Main purpose	Track file changes	Share and collaborate on projects
Location	Runs locally	Provides remote hosting
History	Maintains commits	Displays repository history
Collaboration	Branches and merges	Pull requests, reviews, Issues

Example

Use Git to commit changes on your computer and GitHub to share those commits with your project team.

Exploring the GitHub Interface

A GitHub repository page commonly provides access to:

- **Code:** Project files and directories.
- **Issues:** Bug reports and task discussions.
- **Pull requests:** Proposed changes and reviews.
- **Actions:** Automated workflows and testing.
- **Settings:** Repository permissions and configuration.

Practical Activity

Open a public repository on GitHub and identify its files, commit history, branches, Issues, and pull requests.

Creating a GitHub Account

1. Visit `https://github.com/`.
2. Create an account using an email address.
3. Verify the email address if prompted.
4. Enable two-factor authentication for better security.
5. Sign in and open your profile.

Security Tip

Never share passwords, personal access tokens, or private SSH keys with other people.

Creating a GitHub Repository

1. Sign in to GitHub.
2. Select **New repository**.
3. Enter a repository name, such as `calculator`.
4. Choose public or private visibility.
5. Optionally add a README and `.gitignore`.
6. Select **Create repository**.

Public vs Private

- **Public:** Anyone can view the repository.
- **Private:** Access is limited to authorized users.

Understanding the README File

A README introduces the project and explains how to use it.

Example README.md

```
# Calculator
A simple Python calculator.
## Features
- Addition
- Subtraction
## Run
python main.py
```

A useful README may include:

- Project purpose and features.
- Installation requirements.
- Usage instructions and examples.
- Contributors and licensing information.

Connecting a Local Repository to GitHub

A remote links a local Git repository to a repository hosted on another system.

Add a Remote

```
git remote add origin https://github.com/USERNAME/calculator.git
```

- `git remote add`: Adds a remote connection.
- `origin`: Conventional name for the remote.
- The URL identifies the hosted repository.

Note

Replace `USERNAME` with your GitHub username and use the actual URL of your repository.

Checking Remote Connections

Use the following command to list configured remotes.

Command

```
git remote -v
```

Example output:

```
origin  https://github.com/USERNAME/calculator.git (fetch)
origin  https://github.com/USERNAME/calculator.git (push)
```

- The fetch URL is used to retrieve remote information.
- The push URL is used to send commits.
- A remote can have different fetch and push URLs.

Pushing Code to GitHub

The `git push` command sends local commits to a remote.

Example

```
git branch -M main
git push -u origin main
```

What does this do?

- `git branch -M main` renames the current branch to `main`.
- `git push` sends commits to the remote.
- `-u` sets the upstream tracking relationship.

Prerequisite

The repository must have a configured remote, and you must have the required authentication and permissions.

Cloning a GitHub Repository

Cloning creates a local copy of a remote repository, including its Git history.

Command

```
git clone https://github.com/USERNAME/calculator.git
```

Cloning typically:

- Downloads the repository's files and history.
- Creates a local repository.
- Configures the original repository as a remote named `origin`.
- Sets up local references to remote branches.

Pulling and Fetching Changes

Use these commands to obtain updates from a remote repository.

Fetch

```
git fetch origin
```

Downloads remote updates without integrating them into the current branch.

Pull

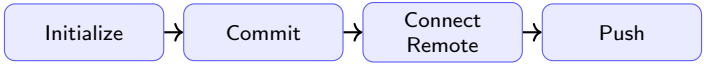
```
git pull
```

Fetches updates and integrates them into the current branch, typically by merging or rebasing according to configuration.

Key Difference

Fetch lets you inspect remote updates first; pull also attempts to integrate them into your current branch.

Complete GitHub Workflow



Typical Commands

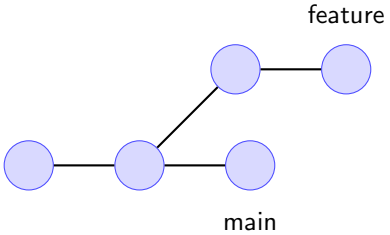
```
git init
git add .
git commit -m "Initial commit"
git remote add origin REPOSITORY_URL
git push -u origin main
```

Outcome: Local commits are published to the configured GitHub repository.

What Is a Branch?

Definition

A branch is an independent line of development that allows changes to be made without directly modifying another branch.



- The main branch contains the primary project history.
- A feature branch isolates new development.
- Branches can later be merged.

Why Use Branches?

Without Branches

- Experimental changes affect main.
- Incomplete features mix with stable code.
- Reviewing individual changes is harder.

With Branches

- Features develop independently.
- Bugs can be fixed separately.
- Changes can be reviewed before merging.

Example

Create a separate branch to implement a login feature while the main branch remains stable.

Creating and Listing Branches

Commands

```
git branch
git branch feature-login
git branch
```

- `git branch` lists local branches.
- `git branch feature-login` creates a branch.
- The current branch is marked with an asterisk.
- Creating a branch does not automatically switch to it.

Remember

A branch is a movable reference to a commit, not a separate copy of the entire project.

Switching Branches

Use `git switch` to change the current branch.

Command

```
git switch feature-login
```

To return to the main branch:

```
git switch main
```

- Git updates the working directory to match the branch.
- Uncommitted changes may prevent switching if they would be overwritten.
- Check the current branch using `git branch`.

Creating and Switching to a Branch

The -c option creates a new branch and switches to it.

Command

```
git switch -c feature-login
```

Typical workflow:

1. Start from the appropriate base branch.
2. Create a feature branch.
3. Edit and test the code.
4. Commit the changes.

Useful Practice

Use descriptive branch names, such as `feature-login`, `fix-navbar`, or `docs-update`.

Merging Branches

Merging integrates changes from one branch into another.

Example

```
git switch main
git merge feature-login
```

1. Switch to the branch that should receive the changes.
2. Run `git merge` with the source branch.
3. Resolve any conflicts if Git reports them.
4. Test the result.

Important

The target branch is the branch currently checked out.

Understanding Merge Conflicts

A merge conflict occurs when Git cannot automatically combine competing changes.

Example

Two developers edit the same line in `main.py`:

- Developer A changes the value to 10.
- Developer B changes the same value to 20.

Why conflicts occur:

- Competing edits to the same lines.
- Changes to a file deleted in another branch.
- Other incompatible file changes.

Remember

A conflict is a request for human or tool-assisted resolution, not necessarily a sign that the repository is damaged.

Resolving Merge Conflicts

Git may mark conflicting sections in a file.

Conflict Markers

```
<<<<<<< HEAD
result = 10
=======
result = 20
>>>>>>> feature-login
```

Resolution steps:

1. Open the conflicting file.
2. Decide which change to keep, or combine both.
3. Remove the conflict markers.
4. Stage the resolved file and complete the merge.

Commands

What Is a Pull Request?

A pull request (PR) proposes changes for review and integration into another branch on a hosting platform such as GitHub.

Typical Pull Request Process

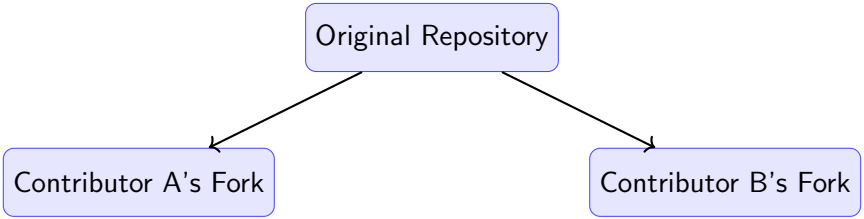
1. Push a feature branch to GitHub.
2. Open a pull request against the target branch.
3. Explain the changes and their purpose.
4. Request reviews and run checks.
5. Address feedback and merge when approved.

Key Point

A pull request supports discussion and review; it is not the same as the local `git pull` command.

Forking a Repository

A fork is a separate repository copy under another account or namespace on a hosting platform.



- Forks support contributions to projects you do not own.
- Contributors can work independently.
- Changes can be proposed through pull requests.

Code Review and Collaboration

Code review helps teams improve code quality before changes are integrated into a shared branch.

Reviewer

- Check correctness.
- Identify potential bugs.
- Suggest improvements.
- Verify tests and readability.

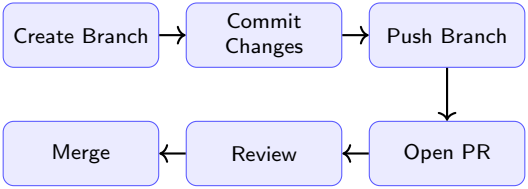
Contributor

- Explain the changes.
- Respond to feedback.
- Update the branch.
- Verify the final result.

Good Practice

Keep pull requests focused, small enough to review, and supported by relevant tests.

Collaborative GitHub Workflow



Takeaway

Branches isolate work, commits record changes, and pull requests support review before integration.

Complete Developer Workflow

1. Create or clone a repository.
2. Create a branch for a new feature.
3. Modify files and test the changes.
4. Stage and commit the changes.
5. Push the branch to GitHub.
6. Open a pull request and request review.
7. Merge the approved changes.

Typical Workflow

Edit → Stage → Commit → Push → Review → Merge

Writing Good Commit Messages

A commit message should clearly describe the purpose of a change.

Poor Messages

- changes
- update
- final code
- bug fixed

Better Messages

- Add login validation
- Fix division by zero
- Update README instructions
- Remove unused functions

Best Practice

Use short, specific messages that explain what the commit does. Keep unrelated changes in separate commits.

Git Tags and Releases

A tag identifies a particular point in repository history, often used to mark a software version.

Useful Commands

```
git tag
git tag -a v1.0.0 -m "Version 1.0.0"
git show v1.0.0
git push origin v1.0.0
```

- Tags identify important commits.
- Semantic versioning commonly uses `MAJOR.MINOR.PATCH`.
- GitHub Releases can attach notes and downloadable files to a version tag.

GitHub Issues

GitHub Issues help teams discuss and track bugs, features, questions, and project tasks.

Common Uses

- Report a software bug.
- Suggest a new feature.
- Assign tasks to contributors.
- Discuss requirements and possible solutions.

Example Issue

Title: Calculator crashes for zero division
Description: Handle division by zero and display an appropriate error message.

GitHub Projects

GitHub Projects provides tools for organizing work and tracking project progress.

Status	Example Task
Todo	Design calculator interface
In Progress	Implement division operation
In Review	Review input validation
Done	Write project README

- Organize work using tables or board views.
- Track progress across tasks.
- Link tasks to Issues and pull requests.

Fork vs Clone

Feature	Fork	Clone
Meaning	Creates a separate hosted repository copy	Creates a local repository copy
Location	GitHub or another hosting platform	Local computer
Purpose	Independent contribution	Local development
Typical use	Contribute to another project	Work on a repository locally

Example

Fork a public project on GitHub, clone your fork, make changes, and propose them to the original project through a pull request.

Merge vs Rebase

Both commands integrate changes, but they handle history differently.

Merge

```
git switch main
git merge feature
```

- Combines branch histories.
- May create a merge commit.
- Preserves existing commits.

Rebase

```
git switch feature
git rebase main
```

- Replays commits on a new base.
- Creates rewritten commit identities.
- Can produce a linear history.

Caution

Avoid rebasing shared commits unless the team understands the consequences of rewriting history.

Common Git Errors and Solutions

Problem	Possible Solution
Untracked files	Stage required files using `git add`.
Push rejected	Fetch remote changes and integrate them.
Merge conflict	Edit conflicting files and complete the merge.
Wrong branch	Check with `git branch`.
Nothing to commit	Inspect `git status` and staged changes.
Authentication failed	Check credentials and repository permissions.

Diagnostic Command

```
git status
```

Authentication and Security

- Use HTTPS or SSH to connect to GitHub.
- Configure a personal access token when required for HTTPS authentication.
- Use SSH keys securely when choosing SSH.
- Never commit passwords, API keys, or private tokens.
- Use a `.gitignore` file for sensitive local configuration files.
- Review repository visibility and access permissions.

Important

Adding a secret to `.gitignore` does not remove it from Git history if it has already been committed.

Git Best Practices

1. Commit changes frequently and logically.
2. Write clear commit messages.
3. Use descriptive branch names.
4. Review changes before committing.
5. Pull or fetch updates regularly.
6. Test changes before merging.
7. Keep credentials and unnecessary generated files out of the repository.
8. Document projects using a README.

Key Principle

A clean repository history makes collaboration, debugging, and maintenance easier.

Mini Project: Version-Controlled Calculator

Objective

Create a simple Python calculator, track its history using Git, and publish the project on GitHub.

Project requirements:

- Create a Python program.
- Initialize a local Git repository.
- Record the initial version in a commit.
- Publish the repository on GitHub.
- Implement a new feature using a branch.
- Submit the feature through a pull request.

Expected Outcome

A GitHub repository containing the calculator code, commit history, and a completed feature workflow.

Step 1: Initialize the Repository

Create a directory and initialize Git.

Commands

```
mkdir calculator
cd calculator
git init
```

Create a Python file named `main.py`:

main.py

```
print("Calculator")
a = 10
b = 5
print("Sum:", a + b)
```

Step 2: Record the First Version

Stage the Python file and create the first commit.

Commands

```
git status
git add main.py
git status
git commit -m "Add initial calculator"
git log --oneline
```

Observe the results:

- The file moves from untracked to staged.
- The commit records the staged version.
- The log displays the new commit.

Learning Outcome

Understand how Git records changes in repository history.

Step 3: Publish the Project on GitHub

Create an empty repository named `calculator` on GitHub. Do not initialize it with a README for this exercise.

Commands

```
git branch -M main
git remote add origin REPOSITORY_URL
git push -u origin main
```

- Replace `REPOSITORY_URL` with your repository URL.
- Complete authentication if prompted.
- Open the GitHub repository to verify the uploaded files.

Expected Result

The calculator source code and its initial commit are visible in the GitHub repository.

Step 4: Implement a Feature Branch

Create a branch for a new calculator feature.

Commands

```
git switch -c feature-subtraction
```

Add subtraction to `main.py`:

New Code

```
print("Difference:", a - b)
```

Stage, commit, and push the feature:

Commands

```
git add main.py
git commit -m "Add subtraction feature"
git push -u origin feature-subtraction
```

Step 5: Create a Pull Request

After pushing the feature branch:

1. Open the repository on GitHub.
2. Select the option to create a pull request.
3. Set the base branch to `main`.
4. Set the compare branch to `feature-subtraction`.
5. Review the changes and create the pull request.
6. Merge the pull request after reviewing the changes.

Verify

Switch back to the local main branch and obtain the merged changes.

Commands

```
git switch main
git pull origin main
```

Thank You!!!!!!